7 / 7 TESTS PASS

NON-UVM

Working SoC RTL prototype

A 5-stage RV32I core runs firmware over a two-master AHB fabric. The data path reaches SRAM, mission registers and APB peripherals. This build is verified in RTL simulation; the sensor is modeled by the testbench.

Integrated data path

RV32I core
instruction + data

→

AHB interconnect
2 masters / 4 slaves

→

Mission + APB
sensor to UART / SPI

Mission pipeline

Event and priority
8 regions; rising-edge events; highest priority wins.

Observe and store
Tagged 32-bit sensor sample; 256-entry FIFO.

Packet and send
18-byte packet; CRC-16; UART or SPI.

Memory and peripheral map

0x8000_0000	**256 KiB boot SRAM**	Program loaded with a hex image
0x4000_0000	**64 KiB data SRAM**	CPU read/write verified
0x5000_0000	**Mission AHB registers**	Configure, trigger, observe, transmit
0x6000_0000	**APB window**	GPIO, UART, SPI, timer, CLINT, PLIC

What the live simulation demonstrates

- Firmware writes and reads GPIO and data SRAM; the imported APB timer raises a compare interrupt.
- Firmware triggers a region observation; the testbench returns a tagged sensor value.
- Telemetry is checked byte-for-byte on the UART pin, then separately on SPI MOSI.
- CLINT and PLIC interrupts enter CPU handlers, report mcause and return with mret.

Evidence and next engineering steps

Self-checking regression

Mission blocks	Priority, FIFO, payload and CRC; two packets	PASS
Full SoC / UART	CPU MMIO, SRAM, timer IRQ, decoded UART bytes	PASS
Full SoC / SPI	CPU observation to SPI mode-0 MOSI bytes	PASS
Interrupt registers	PLIC priority, claim, complete and rearm; CLINT	PASS
CPU software IRQ	MSIP trap, mcause, handler and mret	PASS
CPU external IRQ	GPIO -> PLIC -> MEIP trap, claim and mret	PASS
CPU timer IRQ	MTIP trap, handler and mret	PASS

Honest scope for the meeting

Working in RTL simulation

CPU, AHB/APB, two SRAMs, mission path, UART, SPI, GPIO, timer, PLIC/CLINT and machine interrupt handlers.

Still required

Real sensor acquisition, flash boot, JTAG debug, USB bridge, FPGA pins/timing and hardware validation.

How teammates can reproduce it

Repository github.com/Shivay315/1TOPS-SoC-RTL

Run `bash scripts/run_tests.sh`

Requires Python 3, Icarus Verilog, Verilator and a C++ compiler. The suite generates firmware images and writes detailed logs under build/. No UVM is used.

Suggested presentation wording

"We have a working SoC RTL prototype verified in simulation with seven non-UVM tests. The CPU, bus, memories, mission path, UART/SPI telemetry and interrupts work in the demonstrated scenarios. FPGA and physical interface work is next."